Exercise Sheet 09

The Living Ledger

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active). That installs pandas and openpyxl (for Excel files).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_open_the_file.py`.
- Every exercise starts with the same three lines (the **opener**). Type them, don't paste them:

```
import pandas as pd

pd.set_option("display.width", 120)
df = pd.read_csv("data/thornmere_payroll.csv")
```

- Printed tables must match the expected output exactly, including the numbers down the left (the **index**). If yours differ, you've filtered or sorted differently.
- Hand in: commit and push to `lesson-09`, then open a pull request. Tell Lars which Excel habit you miss, and which you don't.

Exercise 1 Open the file

5 min · *ex1_open_the_file.py*

> **Raven from Mistress Pell (Paymaster):** *"Orrin says pandas opens the whole Roll of the Realm as a 'DataFrame'. Is that a spreadsheet? It sounds like a spreadsheet."*

(a) Print `type(df)`, `df.shape` and `list(df.columns)`.

(b) Print an empty line, then the first five rows of just these columns: employee_id, first_name, last_name, department, annual_salary. Select columns with a list inside the brackets: `df[["a", "b"]]`.

(c) Print an empty line, then the last three rows of employee_id, department, annual_salary (`.tail(3)`).

EXPECTED OUTPUT

```
<class 'pandas.DataFrame'>
(200, 12)
['employee_id', 'first_name', 'last_name', 'gender', 'department', 'job_title', 'grade',
'location', 'weekly_hours', 'annual_salary', 'bonus', 'start_date']

  employee_id first_name   last_name  department  annual_salary
0      TM1092        Nye  Pennyworth       Guard          41900
1      TM1178       Hild    Woolpack     Library          26550
2      TM1105      Maren   Marchbank       Guard          24950
3      TM1180        Una    Wildmoor     Library          29000
4      TM1189       Nell   Oakhollow  Apothecary          31750

    employee_id department  annual_salary
197      TM1008      Forge          50500
198      TM1118      Guard          31150
199      TM1077    Harbour          24550
```

Hint: The numbers down the left are the **index**: row labels, starting at 0. They stick to their rows when you filter or sort.

Exercise 2 — One column at a time

10 min · `ex2_one_column.py`

> **Raven from Mistress Pell:** *"The salary column. Mean, median, max, total. And how many guilds do we actually have? The Library says 8, the Treasury says 9."*

(a) `salary = df["annual_salary"]`. Print its type and `salary.head(3)`.

(b) Print an empty line, then the mean (rounded to 2 dp), median, max and sum, each on its own line.

(c) Print an empty line, then how many different guilds there are (the column is still called `department`) (`.nunique()`) and the sorted list of them (`sorted(... .unique())`).

(d) Print an empty line, then `salary.describe().round(2)`.

EXPECTED OUTPUT

```
<class 'pandas.Series'>
0    41900
1    26550
2    24950
Name: annual_salary, dtype: int64

Mean: 31999.25
Median: 29375.0
Max: 68750
Total: 6399850

Guilds: 8
['Apothecary', 'Clockworks', 'Forge', 'Guard', 'Harbour', 'Kitchens', 'Library',
'Treasury']

count      200.00
mean     31999.25
std      10893.13
min      10300.00
25%      25025.00
50%      29375.00
75%      37312.50
max      68750.00
Name: annual_salary, dtype: float64
```

Hint: A single column is a **Series**. It behaves like last lesson's NumPy array, with an index and a name attached.

Exercise 3 — AutoFilter

10 min · `ex3_autofilter.py`

> **Raven from Lady Siv (Rewards):** *"Everyone over 55,000 crowns. Part-timers in the Guard. How many live away from the harbour. And how many Wardens, because the guild charter says 31 and I don't believe it."*

(a) `high = df[df["annual_salary"] > 55000]`. Print the count (`len(high)`) and the table of first_name, last_name, department, annual_salary. Empty line.

(b) Part-time in the Guard: department is Guard *and* weekly_hours under 37.5. Count, then first_name, last_name, weekly_hours, annual_salary. Empty line.

(c) Count people whose location is Thornmere Keep or Westmarch (`.isin([...])`), people not in Harbourtown (`!=`), and people whose job_title contains "Warden" (`.str.contains(...)`).

EXPECTED OUTPUT

```
Over 55,000 cr: 8
    first_name    last_name  department  annual_salary
15       Silas   Pennyworth       Guard          61750
```

```
84         Ivo  Nightingale    Kitchens          63650
94      Linnet   Quickwater    Treasury          68750
116      Rowan     Woolpack  Clockworks          56750
120     Aldous    Underhill  Clockworks          55450
128      Edric      Redfern  Clockworks          60800
161     Gideon   Greymantle  Clockworks          59150
195    Merrick      Ashdown  Clockworks          59900

Part-time in the Guard: 6
    first_name    last_name  weekly_hours  annual_salary
44        Rafe     Kettleby         22.50          14050
110       Thea      Dunmore         18.75          11200
127      Cerys   Quickwater         30.00          20150
136    Leofric    Saltmarsh         30.00          21450
143       Jory     Woolpack         22.50          14050
168     Hamish  Fairweather         30.00          26900

Keep or Westmarch: 160
Not in Harbourtown: 160
Wardens: 28
```

Hint: Same as NumPy: a comparison gives a mask, the mask goes in square brackets. Combine with `&` and `|`, brackets round each part.

Exercise 4 New columns

10 min · `ex4_new_columns.py`

> **Raven from Mistress Pell:** *"I always add the same helper columns: monthly, hourly, full-time equivalent, salary plus bonus. Every. Single. Moon."*

(a) Add columns: `monthly` (salary / 12, rounded 2 dp), `hourly` (salary / 52 / hours, 2 dp), `fte_salary` (salary scaled to 37.5 hours, rounded to 0 dp), `total_pay` (salary + bonus), `full_time` (hours == 37.5), and `name` (first and last name joined with a space).

(b) Print the shape, then the first five rows of name, monthly, hourly, fte_salary, total_pay, full_time. Empty line.

(c) Print the full-time share (mean of the True/False column), the highest hourly rate, and the total pay bill including bonus.

EXPECTED OUTPUT

```
(200, 18)
              name  monthly  hourly  fte_salary  total_pay  full_time
0   Nye Pennyworth  3491.67   21.49     41900.0      41900       True
1    Hild Woolpack  2212.50   13.62     26550.0      26550       True
2  Maren Marchbank  2079.17   12.79     24950.0      24950       True
3     Una Wildmoor  2416.67   14.87     29000.0      29000       True
4   Nell Oakhollow  2645.83   16.28     31750.0      33490       True

Full-time share: 0.87
Highest hourly rate: 35.26
Total pay bill incl. bonus: 6624220
```

Hint: `df["new"] = ...` creates the column; the right-hand side is column maths like last lesson. Round the whole column: `(df["a"] / 12).round(2)`.

Exercise 5 Sorting

10 min · `ex5_sorting.py`

> **Decree from Queen Aldara:** *"Who are our five best-paid subjects per hour? And the three worst? And can I have it by guild? And who has served longest? (Four questions, I know. I am the Queen.)"*

Add the `hourly` column from Exercise 4, and set `cols = ["first_name", "last_name", "department", "hourly"]`.

(a) "Top 5 hourly rates:" then the five highest (`sort_values("hourly", ascending=False)[cols].head()`). Empty line.

(b) "Bottom 3 hourly rates:" then the three lowest. Empty line.

(c) "By department, then hourly rate (highest first):" then the first four rows sorted by department A–Z and within that by hourly rate, highest first (`ascending=[True, False]`). Empty line.

(d) "Longest serving:" then the three earliest start dates (first_name, last_name, start_date).

EXPECTED OUTPUT

```
Top 5 hourly rates:
    first_name    last_name  department  hourly
94      Linnet   Quickwater    Treasury   35.26
84         Ivo  Nightingale    Kitchens   32.64
7        Cadoc  Nightingale  Apothecary   32.44
104    Leofric   Quickwater    Kitchens   31.79
15       Silas   Pennyworth       Guard   31.67

Bottom 3 hourly rates:
    first_name  last_name department  hourly
55       Freya   Larkspur      Forge   10.26
140        Nye    Ashdown      Forge   10.46
65      Hollis  Saltmarsh      Forge   10.51

By department, then hourly rate (highest first):
    first_name    last_name  department  hourly
7        Cadoc  Nightingale  Apothecary   32.44
174      Nerys         Pike  Apothecary   25.72
148        Mab   Cinderhall  Apothecary   25.44
115      Emrys        Vance  Apothecary   20.13

Longest serving:
    first_name  last_name  start_date
152      Ulric    Ashdown  2006-04-18
157    Merewyn    Redfern  2006-05-06
186       Thea  Hawkridge  2006-10-02
```

Hint: Dates in the format YYYY-MM-DD sort correctly even as text. That's why the Scriptorium uses it. (That, and Orrin insisted.)

Exercise 6 Excel in, CSV out

10 min · `ex6_excel_in_csv_out.py`

> **Raven from Orrin of the Clockworks:** *"The real ledger is the Treasury workbook, `data/payroll.xlsx`, two sheets. Read it. Then write me a CSV of the part-timers so I can stop opening Excel at all."*

No opener this time: you're reading Excel instead of the CSV.

(a) `payroll = pd.read_excel("data/payroll.xlsx", sheet_name="Payroll")`, and the same for the `"Notes"` sheet. Print both shapes, then the last note (`notes["note"].tolist()[-1]`). Empty line.

(b) Filter the part-timers (hours under 37.5), keep only employee_id, first_name, last_name, department, weekly_hours, sort by employee_id, and save with `.to_csv("my_work/part_timers.csv", index=False)`.

(c) Read your new CSV back in and print its shape and first three rows, to prove it worked.

EXPECTED OUTPUT

```
(200, 12)
(5, 1)
The Grimoire of Dalvric reads this sheet. Do not rename it.

(26, 5)
  employee_id first_name    last_name department  weekly_hours
0      TM1007    Rhoswen  Nightingale      Forge          22.5
1      TM1022      Doran      Redfern      Forge          30.0
2      TM1032    Fenwick        Vance      Forge          30.0
```

Hint: `index=False` stops pandas writing the row numbers into the file as a column. Forget it once and you'll have a column called "Unnamed: 0" forever.

Exercise 7 Work anniversaries STRETCH

15 min · ex7_anniversaries.py

> **Raven from Mistress Pell:** *"Every month I work out who has a service anniversary, for the Queen's letters of thanks. By hand. From a column of text dates. Help."*

Opener, then `df["start_date"] = pd.to_datetime(df["start_date"])` and `today = pd.Timestamp("2024-04-05")`.

(a) Add `years_of_service`: `(today - df["start_date"]).dt.days / 365.25`, rounded to 1 dp. Add `start_month` using `.dt.month`.

(b) Filter to people who started in April, sorted by start date. Print the count and the table (first_name, last_name, start_date, years_of_service). Empty line.

(c) Print the average years of service (1 dp) and how many people joined in the last year.

(d) Milestones: people whose years of service rounds to 5, 10 or 15 *and* who started in April. Print the count.

EXPECTED OUTPUT

```
April anniversaries: 12
    first_name    last_name start_date  years_of_service
152      Ulric      Ashdown 2006-04-18              18.0
155     Ninian  Fairweather 2007-04-18              17.0
113      Vesna         Pike 2009-04-27              14.9
187        Siv   Longstride 2010-04-18              14.0
66      Kellan      Dunmore 2012-04-02              12.0
149    Fenwick     Ironfoot 2012-04-28              11.9
70       Osric   Cinderhall 2014-04-21              10.0
105     Hollis     Woolpack 2016-04-17               8.0
87       Osric    Oakhollow 2016-04-24               7.9
6        Ulric     Wildmoor 2019-04-05               5.0
161     Gideon   Greymantle 2020-04-25               3.9
179      Liora   Longstride 2022-04-22               2.0

Average years of service: 9.2
Joined in the last year: 9
April milestones (5/10/15 years): 3
```

Hint: Once a column is a real date, `.dt.month`, `.dt.year` and `.dt.day_name()` all work. Text dates can't do any of that.

Bug hunt Pages from the Grimoire, page 9

15 min · `bughunt/headcount_bug.py`

Dalvric's last ever script: a headcount by town for the Queen, sent by raven at two in the morning with the note "should work". **Three** bugs: one crashes (the error names what it can't find; read it very closely), one gives a wrong number (compare with your Exercise 3), one is a sorting detail.

EXPECTED OUTPUT ONCE FIXED

```
Headcount: 200
Harbourtown headcount: 40
Harbourtown pay bill: 1206300
Highest paid in Harbourtown: Leofric Thistlewood 48450
Lowest paid in Harbourtown: Cerys Quickwater 20150
```

One commit per bug, each message explaining what was wrong and why.